MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report

of

Database Systems Lab (CSS 2212)

TITLE

Digital Forensic Evidence Management System

SUBMITTED

BY

STUDENT NAME	REG. NO	ROLL NO	SECTION
1. Aarya Agarwal	240911190	11	IT D
2. Mahira Srivastava	240911416	24	IT D

School of Computer Engineering

Manipal Institute of Technology, Manipal.

April 2026

SCHOOL OF COMPUTER ENGINEERING

Manipal

07/04/2026

CERTIFICATE

This is to certify that the project titled **Digital Forensic Evidence Management System** is a record of the bonafide work done by **Aarya Agarwal(240911190), Mahira Srivastava(240911416)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in -COMPUTER SCIENCE ENGINEERING(INFORMATION TECHNOLOGY) of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Akshay KC, Assistant Professor – Senior Scale, SCE.**

2. **Dr. Anup Bhat, Assistant Professor, SCE.**

ABSTRACT

The rapid proliferation of digital technologies has significantly increased the role of electronic evidence in criminal investigations, thereby necessitating robust and structured systems for its efficient management. This project presents the design and implementation of a Digital Forensic Evidence Management System (DFEMS) developed using a relational database approach.

The proposed system is designed to systematically manage forensic cases, digital evidence, chain of custody records, forensic analysis, and report generation within a secure and organized framework. It incorporates role-based access control, wherein administrators are authorized to create and manage cases, upload and analyze evidence, and maintain audit logs, while clients are restricted to accessing and interacting only with their respective cases. This ensures both security and controlled data visibility.

The database is implemented using Oracle SQL and comprises multiple interrelated tables structured to ensure data integrity, consistency, and efficient retrieval. Key functionalities include comprehensive evidence tracking, automated audit logging, structured recording of forensic analysis, and generation of detailed investigative reports.

This project effectively demonstrates the practical application of fundamental Database Management System (DBMS) concepts such as entity-relationship modeling, relational schema design, normalization, constraints, triggers, stored procedures, and access control mechanisms. The system ensures secure handling of sensitive forensic data while maintaining transparency, traceability, and accountability, which are critical in real-world investigative environments.

ACM Taxonomy:
[Information Systems]: Database Management Systems; Data Integrity; Access Control; Digital Forensics

SDG:

[SDG 16]: Peace, Justice, and Strong Institutions

Table of Contents

CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

CHAPTER 6: RESULTS & SNAPSHOTS

List of Tables

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER / ERD	Entity Relationship / Entity Relationship Diagram
FK	Foreign Key
NF	Normal Form
PK	Primary Key
PL/SQL	Procedural Language / Structured Query Language
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface

CHAPTER 1: INTRODUCTION

The rapid advancement of digital technologies has significantly transformed the nature of criminal investigations, with a substantial portion of evidence now existing in digital formats such as images, videos, emails, and system logs. The handling of such digital evidence requires a structured, secure, and reliable system to ensure its proper collection, storage, analysis, and presentation.

In traditional systems, the absence of a centralized and well-organized database often leads to challenges such as data redundancy, lack of traceability, unauthorized access, and inefficiencies in managing forensic records. These limitations can compromise the integrity and admissibility of evidence in legal proceedings.

To address these challenges, this project proposes the design and implementation of a Digital Forensic Evidence Management System (DFEMS). The system provides a centralized platform for managing forensic cases and associated digital evidence. It ensures that all activities related to evidence handling are systematically recorded, thereby maintaining the chain of custody and ensuring data integrity.

The proposed system supports role-based access control, allowing administrators to perform operations such as case creation, evidence upload, analysis, and audit logging, while restricting clients to accessing only their respective case information. This enhances both security and controlled accessibility of sensitive data. Furthermore, the system models real-world forensic workflows, including case registration, evidence collection, storage management, forensic analysis, and report generation. By leveraging a relational database approach, the system ensures consistency, minimizes redundancy, and enables efficient data retrieval and reporting.
Thus, this project aims to develop a robust and scalable database system that facilitates efficient digital forensic investigation while ensuring transparency, accountability, and reliability.

1.1 Related Concepts and Background

A Digital Forensic Evidence Management System is a transaction-intensive system that involves multiple users interacting with a shared dataset. Such systems rely heavily on relational database management systems to maintain data consistency, enforce constraints, and support structured data operations.

The project incorporates several fundamental concepts of Database Management Systems (DBMS), which are essential for designing a reliable and efficient system:

- Entity-Relationship Modeling
- Relational Schema Design
- Normalization
- Primary and Foreign Key Constraints
- Triggers and Stored Procedures
- Access Control Mechanisms

Additionally, the concept of **chain of custody** plays a crucial role in digital forensics. It ensures that every action performed on evidence is recorded, thereby preserving its authenticity and making it admissible in legal contexts.

1.2 Overview of the System

The proposed system operates through a structured workflow involving multiple components and user roles. Users can log in either as administrators or clients, with access privileges determined by their assigned roles.

Administrators have the ability to create and manage cases, upload and verify evidence, perform forensic analysis, generate reports, and monitor system activities through audit logs. Clients, on the other hand, have restricted access and can only view their respective cases, upload additional evidence, and perform limited analysis.

The system integrates multiple entities such as cases, users, evidence, analysis results, forensic tools, and audit logs, all interconnected through a relational database schema. This structured design ensures efficient data management and supports seamless execution of forensic workflows.

CHAPTER 2:
PROBLEM STATEMENT & OBJECTIVES

This chapter outlines the key challenges associated with managing digital forensic data and highlights the need for a structured database-driven solution. It defines the problem addressed by the proposed system and presents the primary objectives guiding its design and implementation. The chapter establishes the foundation for developing a secure, efficient, and reliable forensic evidence management system.

2.1 Problem Statement

Modern forensic investigations involve handling large volumes of digital evidence that must be securely stored, tracked, and analyzed. In the absence of a structured database system, several challenges arise:

- Lack of proper tracking of evidence handling (chain of custody)
- Unauthorized access to sensitive case information
- Data redundancy and inconsistency
- Difficulty in generating structured forensic reports
- Absence of audit trails for accountability

Hence, there is a need for a secure, role-based, and normalized database system that can efficiently manage digital forensic data while ensuring integrity and traceability.

2.2 Objectives

The primary objectives of this project are:

- To design a relational database for managing forensic cases and digital evidence
- To implement role-based access control for admin and client users
- To maintain a chain of custody for tracking evidence handling
- To store and manage forensic analysis results
- To implement audit logging for system activities
- To ensure data integrity using constraints and normalization
- To support report generation for forensic investigations

CHAPTER 3:
METHODOLOGY

This chapter describes the design and implementation approach of the proposed Digital Forensic Evidence Management System (DFEMS). It outlines the scope of the system, functional and non-functional requirements, and the overall system architecture. The methodology focuses on how the system models real-world forensic workflows using a structured relational database.

3.1 Scope of the System

The scope of the proposed system includes the development of a database-driven platform to manage digital forensic investigations in a structured and secure manner. The system facilitates the storage, tracking, and analysis of digital evidence associated with various cases.

The system supports user authentication with role-based access, allowing administrators to manage cases, upload evidence, perform forensic analysis, and generate reports, while clients are restricted to accessing only their respective cases. The database design ensures efficient handling of multiple interconnected entities such as users, cases, evidence, analysis results, and audit logs.

However, the current system is limited to database implementation and basic interaction. Advanced features such as automated PDF report generation, integration with external forensic tools, and large-scale deployment are not included in the current scope.

3.2 System Analysis and Requirements

3.2.1Functional Requirements

The functional requirements define the core operations that the system must perform:

a) User Management

- Users can log in as administrators or clients
- Role-based access determines system permissions
- User details are securely stored and managed

b) Case Management

- Administrators can create and manage forensic cases
- Each case is assigned a unique identifier
- Case status is tracked (Open, Under Investigation, Closed)

c) Evidence Management

- Digital evidence such as images, videos, and documents can be uploaded
- Each evidence is linked to a specific case
- Metadata such as file name, type, size, and hash values are stored

d) Chain of Custody

- Every action performed on evidence is recorded
- Tracks handling, transfer, and access of evidence
- Ensures integrity and authenticity of forensic data

e) Forensic Analysis

- Evidence can be analyzed using specific forensic tools
- Analysis results and findings are stored
- Status of analysis is tracked

f) Report Generation

- Structured forensic reports are generated
- Reports include case details, evidence summary, and analysis results
- (Current limitation: PDF export not fully implemented)

g) Audit Logging

- All system activities are recorded
- Tracks user actions such as login, updates, and analysis
- Ensures accountability and system transparency

3.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes of the system:

a) Security

- Role-based access control restricts unauthorized usage
- Sensitive forensic data is protected

b) Data Integrity

- Use of primary and foreign keys ensures valid relationships
- Constraints prevent invalid data entry
- Hash values ensure evidence authenticity

c) Performance

- Efficient querying enables fast retrieval of case and evidence data
- Optimized schema reduces redundancy

d) Scalability

- Designed to support multiple cases and large volumes of evidence
- Can be extended for larger forensic systems

e) Reliability

- Consistent data storage and retrieval
- Use of triggers and procedures ensures correct operations

f) Maintainability

- Modular database design
- Easy to update and extend

3.3 System Design

3.3.1 Major Entities

The system consists of the following major entities:

1. **Case Management Entities**

- **CASES**:
  This is the central entity of the system, storing details of each forensic case such as case number, date, description, location, and status. Each case is uniquely identified and linked to the user who created it.
- **CASE_TYPE**:
  This entity categorizes cases into different types such as cybercrime, financial fraud, identity theft, etc., enabling better classification and filtering of cases.
- **CASE_STATUS**:
  This entity maintains the current status of each case (e.g., Open, Under Investigation, Closed), allowing tracking of case progress.

2. **User and Organizational Entities**

- **USERS**:
  Stores information about all system users, including investigators, analysts, and administrators. Each user is associated with a specific role and department.
- **ROLE**:
  Defines the different roles within the system such as Admin, Investigator, Forensic Analyst, Supervisor, and Viewer. Role-based access control is enforced using this entity.
- **DEPARTMENT**:
  Represents organizational units such as Cybercrime Unit, Forensic Lab, etc., helping in categorizing users and managing responsibilities.

3. **Evidence Management Entities**

- **EVIDENCE**:
  Stores details of digital evidence associated with cases, including file name, type,

size, upload date, and hash values for integrity verification. Each evidence record is linked to a specific case.

- **EVIDENCE_TYPE**:
  Categorizes evidence into types such as images, videos, documents, emails, and network captures.
- **STORAGE_LOCATION**:
  Maintains information about where evidence is stored, such as secure servers, cloud storage, or physical lockers.

4. Chain of Custody and Action Tracking

- **CHAIN_OF_CUSTODY**:
  Records all actions performed on evidence, including who accessed or modified it and when. This entity is critical for maintaining the integrity and authenticity of evidence.
- **ACTION_TYPE**:
  Defines the types of actions that can be performed on evidence, such as uploaded, accessed, transferred, analyzed, and verified.

5. Analysis and Reporting Entities

- **ANALYSIS_RESULTS**:
  Stores the results of forensic analysis performed on evidence, including findings, analysis date, status, and the tool used.
- **FORENSIC_TOOL**:
  Contains information about forensic tools used in analysis, such as Autopsy, Wireshark, and Volatility.
- **FORENSIC_REPORT**:
  Stores generated reports for cases, including report content, type (preliminary, interim, final), and creation details.

6. **System Monitoring and Logging**

- **AUDIT_LOG**:
  Maintains a record of all system activities such as user actions, case updates, evidence uploads, and report generation. This ensures accountability and system transparency.

In total, the database consists of **15 interrelated tables** designed to represent different aspects of the digital forensic investigation process. These entities are connected through primary and foreign key constraints, ensuring referential integrity and enabling efficient data retrieval.

3.3.2 System Workflow

The working of the system follows a structured sequence:

1. User logs into the system
2. Administrator creates a case
3. Evidence is uploaded and stored
4. Chain of custody records all actions
5. Evidence is analyzed using forensic tools
6. Analysis results are stored
7. Reports are generated
8. Audit logs track all activities

This workflow ensures a complete and traceable forensic investigation process.

3.4 Database Implementation Approach

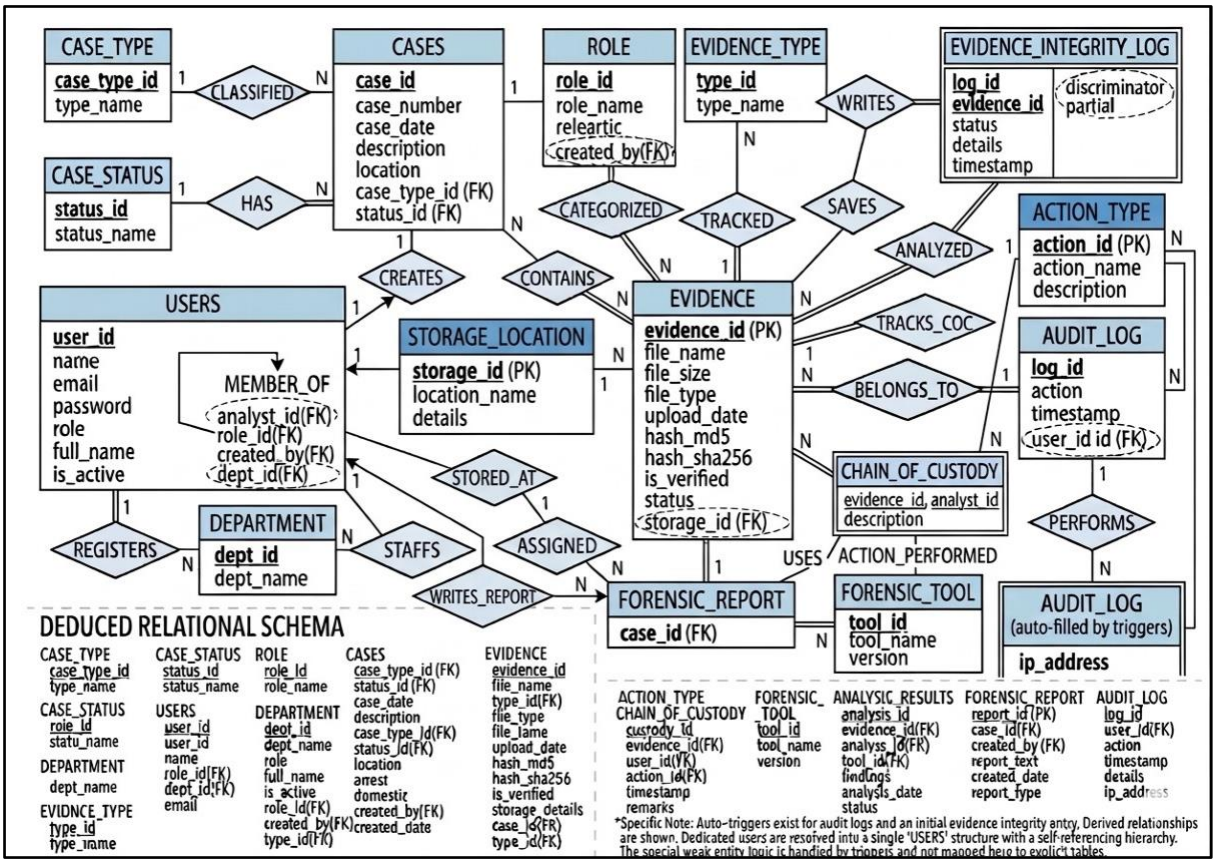
The system is implemented using Oracle SQL and follows a relational database model. The database consists of multiple interrelated tables designed using normalization techniques to eliminate redundancy and ensure efficient data storage.

Advanced database features such as triggers, stored procedures, and functions are used to automate operations like audit logging, evidence tracking, and report generation. These features enhance the reliability and practicality of the system.

CHAPTER 4:

ER Diagram, Relational Tables along with sample data, Normalization

1.1.ER Diagram



1.2. Relation Schema

Entity	Attributes (PK / FK marked)
CASE_TYPE	case_type_id (PK), type_name
CASE_STATUS	status_id (PK), status_name
DEPARTMENT	dept_id (PK), dept_name
ROLE	role_id (PK), role_name, created_by (FK)
USERS	user_id (PK), name, email, password, role_id (FK), dept_id (FK), created_by (FK), full_name, is_active
CASES	case_id (PK), case_number, case_date, description, location, case_type_id (FK), status_id (FK), created_by (FK)
EVIDENCE_TYPE	type_id (PK), type_name
STORAGE_LOCATION	storage_id (PK), location_name, details
EVIDENCE	evidence_id (PK), file_name, file_size, file_type, upload_date, hash_md5, hash_sha256, is_verified, status, type_id (FK), storage_id (FK), case_id (FK)
CHAIN_OF_CUSTODY	custody_id (PK), evidence_id (FK), user_id (FK), action_id (FK), timestamp, remarks
ACTION_TYPE	action_id (PK), action_name, description
FORENSIC_TOOL	tool_id (PK), tool_name, version
FORENSIC_REPORT	report_id (PK), case_id (FK), created_by (FK), report_text, created_date, report_type

ANALYSIS_RESULTS	analysis_id (PK), evidence_id (FK), analyst_id (FK), tool_id (FK), findings, analysis_date, status
AUDIT_LOG	log_id (PK), user_id (FK), action, timestamp, details, ip_address
EVIDENCE_INTEGRITY_LOG	log_id (PK), evidence_id (FK), status, details, timestamp

Table 1: Relation Schema showing entities and attributes

1.3. Normalization

a. First Normal Form

A relation is said to be in First Normal Form if:

- All attributes contain atomic (indivisible) values
- There are no repeating groups or multi-valued attributes

Application in DFEMS

Initially, a forensic system could store case-related data in a single unstructured table, for example-

CASE(case_id, case_number, evidence_files, evidence_types, analyst_names, tools_used)

This structure leads to:

- Multiple values stored in a single attribute
- Repetition of data
- Difficulty in querying

To achieve **1NF**, the data is decomposed into separate tables such as:

- CASES
- EVIDENCE
- USERS
- FORENSIC_TOOL

Each attribute now stores atomic values, and repeating groups are removed.

Result: All tables in the system satisfy 1NF as each field contains only single, indivisible values.

b. Second Normal Form

A relation is in Second Normal Form if:

- It is already in 1NF
- All non-key attributes are fully functionally dependent on the primary key

Application in DFEMS

In the system, tables with composite dependencies are carefully decomposed.

For example:

- In the EVIDENCE table:

  evidence_id → file_name, file_size, file_type, hash_md5

  All attributes depend entirely on the primary key (evidence_id).

- In CHAIN_OF_CUSTODY:

  custody_id → evidence_id, user_id, action_id, timestamp

  No partial dependencies exist, as:

- Each table uses a single primary key
- Non-key attributes depend fully on that key

Result: All relations are in 2NF, eliminating partial dependency issues.

c. Third Normal Form

A relation is in Third Normal Form if:

- It is in 2NF
- There are no transitive dependencies (non-key attributes should not depend on other non-key attributes)

Application in DFEMS

To eliminate transitive dependencies, the system separates related attributes into distinct tables.

Before normalization:

USERS(user_id, name, role_name, dept_name)

Here:

- role_name depends on role_id
- dept_name depends on dept_id

After normalization:

- USERS(user_id, name, role_id, dept_id)
- ROLE(role_id, role_name)
- DEPARTMENT(dept_id, dept_name)

Similarly:

- CASE_TYPE and CASE_STATUS are separated from CASES
- EVIDENCE_TYPE is separated from EVIDENCE
- ACTION_TYPE is separated from CHAIN_OF_CUSTODY

This eliminates redundancy and ensures:

- Consistency of data
- Efficient updates
- Reduced storage duplication

Result: All tables in the system satisfy 3NF.

d. Benefits of Normalisation

The normalization process provides several advantages:

- **Elimination of Redundancy:**
  Data is stored only once, reducing duplication across tables.
- **Improved Data Integrity:**
  Constraints and relationships ensure consistency of data.
- **Efficient Data Retrieval:**
  Structured tables enable optimized queries.

- **Prevention of Anomalies:**

  Avoids insertion, deletion, and update anomalies.
- **Scalability:**

  The system can easily accommodate new cases, evidence, and users without structural changes.

CHAPTER 5: DDL commands & PL/SQL Procedures / Functions / Triggers

5.1 Table Creation

5.1.1 CASE_TYPE

```
CREATE TABLE CASE_TYPE (
   case_type_id   NUMBER PRIMARY KEY,
   type_name      VARCHAR2(100) NOT NULL UNIQUE
);
```

5.1.2 CASE_STATUS

```
CREATE TABLE CASE_STATUS (
   status_id    NUMBER PRIMARY KEY,
   status_name  VARCHAR2(50) NOT NULL UNIQUE
);
```

5.1.3 ROLE

```
CREATE TABLE ROLE (
   role_id    NUMBER PRIMARY KEY,
   role_name  VARCHAR2(100) NOT NULL UNIQUE
);
```

5.1.4 DEPARTMENT

```
CREATE TABLE DEPARTMENT (
   dept_id    NUMBER PRIMARY KEY,
   dept_name  VARCHAR2(100) NOT NULL UNIQUE
);
```

5.1.5 USERS

```
CREATE TABLE USERS (
   user_id   NUMBER PRIMARY KEY,
   name      VARCHAR2(100) NOT NULL,
   role_id   NUMBER NOT NULL,
   dept_id   NUMBER NOT NULL,
   email     VARCHAR2(150) UNIQUE NOT NULL,
   CONSTRAINT fk_user_role FOREIGN KEY (role_id) REFERENCES
ROLE(role_id),
   CONSTRAINT fk_user_dept FOREIGN KEY (dept_id) REFERENCES
DEPARTMENT(dept_id)
);
```

5.1.6 CASES

```
CREATE TABLE CASES (
   case_id       NUMBER PRIMARY KEY,
   case_number   VARCHAR2(50) NOT NULL UNIQUE,
   case_date     DATE NOT NULL,
   description   VARCHAR2(1000),
   case_type_id  NUMBER NOT NULL,
   status_id     NUMBER NOT NULL,
   location      VARCHAR2(200),
   arrest        CHAR(1) DEFAULT 'N' CHECK (arrest IN ('Y','N')),
   domestic      CHAR(1) DEFAULT 'N' CHECK (domestic IN ('Y','N')),
   created_by    NUMBER,
   created_date  DATE DEFAULT SYSDATE,
   CONSTRAINT fk_case_type   FOREIGN KEY (case_type_id) REFERENCES
CASE_TYPE(case_type_id),
   CONSTRAINT fk_case_status FOREIGN KEY (status_id)    REFERENCES
CASE_STATUS(status_id),
   CONSTRAINT fk_case_user   FOREIGN KEY (created_by)   REFERENCES
USERS(user_id)
);
```

5.1.7 EVIDENCE_TYPE

```
CREATE TABLE EVIDENCE_TYPE (
type_id    NUMBER PRIMARY KEY,
type_name  VARCHAR2(100) NOT NULL UNIQUE
);
```

5.1.8 STORAGE_LOCATION

```
CREATE TABLE STORAGE_LOCATION (
  storage_id     NUMBER PRIMARY KEY,
  location_name  VARCHAR2(200) NOT NULL,
  details        VARCHAR2(500)
);
```

5.1.9 EVIDENCE

```
CREATE TABLE EVIDENCE (
  evidence_id   NUMBER PRIMARY KEY,
  case_id       NUMBER NOT NULL,
  type_id       NUMBER NOT NULL,
  storage_id    NUMBER NOT NULL,
  file_name     VARCHAR2(300) NOT NULL,
  file_size     NUMBER,          -- in bytes
  file_type     VARCHAR2(50),
  upload_date   DATE DEFAULT SYSDATE,
  hash_md5      VARCHAR2(64),    -- for integrity checking
  hash_sha256   VARCHAR2(128),
  is_verified   CHAR(1) DEFAULT 'N' CHECK (is_verified IN ('Y','N')),
  CONSTRAINT fk_ev_case    FOREIGN KEY (case_id)    REFERENCES
CASES(case_id),
  CONSTRAINT fk_ev_type    FOREIGN KEY (type_id)    REFERENCES
EVIDENCE_TYPE(type_id),
  CONSTRAINT fk_ev_storage FOREIGN KEY (storage_id) REFERENCES
STORAGE_LOCATION(storage_id));
```

5.1.10 ACTION_TYPE

```
CREATE TABLE ACTION_TYPE (
  action_id    NUMBER PRIMARY KEY,
  action_name  VARCHAR2(100) NOT NULL UNIQUE
);
```

5.1.11 CHAIN_OF_CUSTODY

```
CREATE TABLE CHAIN_OF_CUSTODY (
custody_id   NUMBER PRIMARY KEY,
evidence_id  NUMBER NOT NULL,
user_id      NUMBER NOT NULL,
action_id    NUMBER NOT NULL,
timestamp    TIMESTAMP DEFAULT SYSTIMESTAMP,
remarks      VARCHAR2(500),
CONSTRAINT fk_coc_evidence FOREIGN KEY (evidence_id) REFERENCES
EVIDENCE(evidence_id),
CONSTRAINT fk_coc_user FOREIGN KEY (user_id) REFERENCES
USERS(user_id),
CONSTRAINT fk_coc_action FOREIGN KEY (action_id) REFERENCES
ACTION_TYPE(action_id)
);
```

5.1.12 FORENSIC_TOOL

```
CREATE TABLE FORENSIC_TOOL (
  tool_id    NUMBER PRIMARY KEY,
  tool_name  VARCHAR2(150) NOT NULL,
  version    VARCHAR2(50)
);
```

5.1.13 ANALYSIS_RESULTS

```
CREATE TABLE ANALYSIS_RESULTS (
  analysis_id    NUMBER PRIMARY KEY,
  evidence_id    NUMBER NOT NULL,
  analyst_id     NUMBER NOT NULL,
  tool_id        NUMBER NOT NULL,
  findings       CLOB,
  analysis_date  DATE DEFAULT SYSDATE,
  status         VARCHAR2(50) DEFAULT 'PENDING'
                 CHECK (status IN
('PENDING','IN_PROGRESS','COMPLETE','REVIEWED')),
  CONSTRAINT fk_ar_evidence FOREIGN KEY (evidence_id) REFERENCES
EVIDENCE(evidence_id),
  CONSTRAINT fk_ar_analyst  FOREIGN KEY (analyst_id)  REFERENCES
USERS(user_id),
  CONSTRAINT fk_ar_tool     FOREIGN KEY (tool_id)     REFERENCES
FORENSIC_TOOL(tool_id));
```

5.1.14 FORENSIC_REPORT

```
CREATE TABLE FORENSIC_REPORT (
   report_id     NUMBER PRIMARY KEY,
   case_id       NUMBER NOT NULL,
   created_by    NUMBER NOT NULL,
   report_text   CLOB,
   created_date  DATE DEFAULT SYSDATE,
   report_type   VARCHAR2(50) DEFAULT 'PRELIMINARY'
                 CHECK (report_type IN ('PRELIMINARY','INTERIM','FINAL')),
   CONSTRAINT fk_fr_case FOREIGN KEY (case_id)    REFERENCES
CASES(case_id),
   CONSTRAINT fk_fr_user FOREIGN KEY (created_by) REFERENCES
USERS(user_id));
```

5.1.15 AUDIT_LOG

```
CREATE TABLE AUDIT_LOG (
   log_id     NUMBER PRIMARY KEY,
   user_id    NUMBER,
   action     VARCHAR2(200) NOT NULL,
   timestamp  TIMESTAMP DEFAULT SYSTIMESTAMP,
   details    VARCHAR2(1000),
   ip_address VARCHAR2(50),
   CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES
USERS(user_id)
);
```

5.2 Triggers

5.2.1 Auto-assign PK for CASE_TYPE

```
CREATE OR REPLACE TRIGGER trg_case_type_pk
BEFORE INSERT ON CASE_TYPE
FOR EACH ROW
BEGIN
   IF :NEW.case_type_id IS NULL THEN
      SELECT seq_case_type.NEXTVAL INTO :NEW.case_type_id FROM DUAL;
   END IF;
END;
/
```

5.2.2 Audit trigger on CASES (track status changes)

```
CREATE OR REPLACE TRIGGER trg_case_status_change
AFTER UPDATE OF status_id ON CASES
FOR EACH ROW
BEGIN
  INSERT INTO AUDIT_LOG(log_id, user_id, action, timestamp, details)
  VALUES(
    seq_audit.NEXTVAL,
    :NEW.created_by,
    'CASE_STATUS_CHANGED',
    SYSTIMESTAMP,
    'Case ' || :NEW.case_number || ': Status changed from ' || :OLD.status_id || ' to '
|| :NEW.status_id
  );
END;
/
```

5.2.3 Auto create initial Chain of Custody on evidence upload

```
CREATE OR REPLACE TRIGGER trg_evidence_initial_custody
AFTER INSERT ON EVIDENCE
FOR EACH ROW
DECLARE
  v_upload_action NUMBER;
BEGIN
  SELECT action_id INTO v_upload_action FROM ACTION_TYPE WHERE
action_name = 'UPLOADED' AND ROWNUM = 1;
  INSERT INTO CHAIN_OF_CUSTODY(custody_id, evidence_id, user_id,
action_id, timestamp, remarks)
  VALUES(seq_custody.NEXTVAL, :NEW.evidence_id, 1, v_upload_action,
SYSTIMESTAMP, 'Initial upload of evidence file: ' || :NEW.file_name);
EXCEPTION
  WHEN NO_DATA_FOUND THEN NULL; -- Skip if action type not yet seeded
END;
/
```

5.3 Functions

5.3.1 Get total evidence count for a case

```
CREATE OR REPLACE FUNCTION fn_evidence_count(p_case_id IN NUMBER)
RETURN NUMBER IS
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count FROM EVIDENCE WHERE case_id =
p_case_id;
   RETURN v_count;
END;
/
```

5.3.2 Get case status name by case_id

```
CREATE OR REPLACE FUNCTION fn_get_case_status(p_case_id IN NUMBER)
RETURN VARCHAR2 IS
   v_status VARCHAR2(50);
BEGIN
   SELECT cs.status_name INTO v_status
   FROM CASES c JOIN CASE_STATUS cs ON c.status_id = cs.status_id
   WHERE c.case_id = p_case_id;
   RETURN v_status;
EXCEPTION
   WHEN NO_DATA_FOUND THEN RETURN 'NOT FOUND';
END;
/
```

5.3.3 Validate email format

```
CREATE OR REPLACE FUNCTION fn_validate_email(p_email IN VARCHAR2)
RETURN NUMBER IS  -- Returns 1 valid, 0 invalid
BEGIN
   IF REGEXP_LIKE(p_email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-
z]{2,}$')    THEN
      RETURN 1;
   ELSE
      RETURN 0;
   END IF;
END;
/
```

5.4 Views

5.4.1 Full case details view

```
CREATE OR REPLACE VIEW vw_case_details AS
SELECT
   c.case_id, c.case_number, c.case_date, c.description,
   ct.type_name AS case_type, cs.status_name AS case_status,
   c.location, c.arrest, c.domestic,
   u.name AS created_by,
   fn_evidence_count(c.case_id) AS evidence_count
FROM CASES c
JOIN CASE_TYPE ct   ON c.case_type_id = ct.case_type_id
JOIN CASE_STATUS cs ON c.status_id = cs.status_id
LEFT JOIN USERS u   ON c.created_by = u.user_id;
```

5.4.2 Evidence with case info

```
CREATE OR REPLACE VIEW vw_evidence_details AS
SELECT
   e.evidence_id, e.file_name, e.file_type,
   fn_format_file_size(e.file_size) AS file_size,
   e.upload_date, e.is_verified, e.hash_md5,
   et.type_name AS evidence_type,
   sl.location_name AS storage_location,
   c.case_number, c.case_id
FROM EVIDENCE e
JOIN EVIDENCE_TYPE et     ON e.type_id = et.type_id
JOIN STORAGE_LOCATION sl  ON e.storage_id = sl.storage_id
JOIN CASES c              ON e.case_id = c.case_id;
```

5.4.3 Analysis summary

```
CREATE OR REPLACE VIEW vw_analysis_summary AS
SELECT
   ar.analysis_id, ar.analysis_date, ar.status,
   ar.findings,
   u.name AS analyst_name,
   ft.tool_name, ft.version AS tool_version,
   e.file_name AS evidence_file,
   c.case_number
FROM ANALYSIS_RESULTS ar
JOIN USERS u         ON ar.analyst_id = u.user_id
JOIN FORENSIC_TOOL ft ON ar.tool_id = ft.tool_id
JOIN EVIDENCE e      ON ar.evidence_id = e.evidence_id
JOIN CASES c         ON e.case_id = c.case_id;
```

5.5 Insert Statements(Any one)

5.5.1 Users

```
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Admin System',  1, 4,
'admin@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Arjun Sharma',  2, 1,
'arjun.sharma@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Priya Nair',    3, 3,
'priya.nair@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Rahul Singh',   2, 2,
'rahul.singh@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Meena Pillai',  4, 4,
'meena.pillai@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Vikram Das',    3, 3,
'vikram.das@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Sunita Rao',    5, 5,
'sunita.rao@dfems.gov');
INSERT INTO USERS(name, role_id, dept_id, email) VALUES('Deepak Kumar',  2, 1,
'deepak.kumar@dfems.gov');
```

5.6 Procedures(Any one)

5.6.1 Upload evidence

```
CREATE OR REPLACE PROCEDURE sp_upload_evidence(
   p_case_id     IN NUMBER,
   p_type_id     IN NUMBER,
   p_storage_id  IN NUMBER,
   p_file_name   IN VARCHAR2,
   p_file_size   IN NUMBER,
   p_file_type   IN VARCHAR2,
   p_hash_md5    IN VARCHAR2,
   p_uploaded_by IN NUMBER,
   p_evidence_id OUT NUMBER
) IS
BEGIN
   INSERT INTO EVIDENCE(case_id, type_id, storage_id, file_name, file_size, file_type,
hash_md5, upload_date)
   VALUES(p_case_id, p_type_id, p_storage_id, p_file_name, p_file_size, p_file_type,
p_hash_md5, SYSDATE)
   RETURNING evidence_id INTO p_evidence_id;

   INSERT INTO AUDIT_LOG(user_id, action, timestamp, details)
   VALUES(p_uploaded_by, 'EVIDENCE_UPLOADED', SYSTIMESTAMP,
        'Evidence uploaded: ' || p_file_name || ' for case_id: ' || p_case_id);
   COMMIT;
EXCEPTION
   WHEN OTHERS THEN ROLLBACK; RAISE;
END;
/
```

5.7 SQL Queries

5.7.1 All open cases with evidence count

```
SELECT
   c.case_number, ct.type_name, cs.status_name,
   c.location, c.case_date,
   fn_evidence_count(c.case_id) AS evidence_count
FROM CASES c
JOIN CASE_TYPE ct   ON c.case_type_id = ct.case_type_id
JOIN CASE_STATUS cs ON c.status_id = cs.status_id
WHERE cs.status_name != 'CLOSED'
ORDER BY c.case_date DESC;
```

5.7.2 Chain of custody for specific evidence

```
SELECT
   u.name AS handled_by, at.action_name AS action,
   coc.timestamp, coc.remarks
FROM CHAIN_OF_CUSTODY coc
JOIN USERS u       ON coc.user_id = u.user_id
JOIN ACTION_TYPE at ON coc.action_id = at.action_id
WHERE coc.evidence_id = 5001
ORDER BY coc.timestamp;
```

5.7.3 All evidence for a case with type and storage

```
SELECT
   e.evidence_id, e.file_name, et.type_name,
   fn_format_file_size(e.file_size) AS size,
   sl.location_name, e.upload_date, e.is_verified
FROM EVIDENCE e
JOIN EVIDENCE_TYPE et    ON e.type_id = et.type_id
JOIN STORAGE_LOCATION sl ON e.storage_id = sl.storage_id
WHERE e.case_id = 1001;
```

CHAPTER 6:
RESULTS AND SNAPSHOTS

1.1.Expected Outcomes

The implementation of the Digital Forensic Evidence Management System (DFEMS) is expected to provide a structured and efficient platform for managing digital forensic investigations. The system successfully demonstrates how database management principles can be applied to handle complex, real-world scenarios involving sensitive data.
The expected outcomes of the system are as follows:

- Centralized Data Management:
  All case-related information, including evidence, analysis results, and reports, is stored in a centralized relational database, ensuring easy access and management.
- Efficient Evidence Tracking:
  The system enables systematic tracking of digital evidence through unique identifiers and metadata such as file type, size, and hash values, ensuring proper organization.
- Maintenance of Chain of Custody:
  Every action performed on evidence is recorded in the Chain of Custody table, ensuring traceability and preserving the integrity of evidence for legal purposes.
- Role-Based Access Control:
  The system restricts access based on user roles, allowing administrators full control while limiting clients to their respective case data, thereby enhancing security.
- Automated Logging and Monitoring:
  Audit logs are automatically generated for critical operations, ensuring accountability and transparency in system usage.

- Structured Forensic Analysis:
  The system allows recording of analysis results along with tools used and findings, enabling organized documentation of investigative processes.
- Report Generation Capability:
  The system supports generation of structured forensic reports containing case details, evidence summaries, and analysis findings (with scope for further enhancement in PDF generation).
- Data Integrity and Consistency:
  Use of primary keys, foreign keys, constraints, and triggers ensures accurate and consistent data storage across the system.

1.2.Results and Discussion

The developed system demonstrates the successful application of relational database concepts in managing digital forensic data. The results indicate that the system is capable of handling multiple cases, users, and evidence records efficiently while maintaining data integrity and security.

The database design effectively reduces redundancy through normalization and ensures consistency through the use of constraints and relationships. The separation of entities such as users, roles, evidence types, and case statuses allows for better organization and scalability of the system.

The implementation of triggers and stored procedures enhances automation within the system. For instance, automatic logging of actions and validation mechanisms ensure that important operations are recorded without manual intervention. This contributes to improved reliability and reduces the chances of human error.

The Chain of Custody mechanism plays a crucial role in the system by maintaining a detailed record of all interactions with evidence. This feature ensures transparency and supports the admissibility of evidence in legal contexts.

Furthermore, the system demonstrates effective access control through role-based restrictions, ensuring that sensitive information is accessible only to authorized users. This is particularly important in forensic systems where data confidentiality is critical.

Overall, the results indicate that the system is reliable, scalable, and capable of supporting digital forensic workflows in a structured and efficient manner. The project successfully bridges theoretical DBMS concepts with practical implementation, demonstrating its applicability in real-world scenarios.

1.3.Application Screenshots

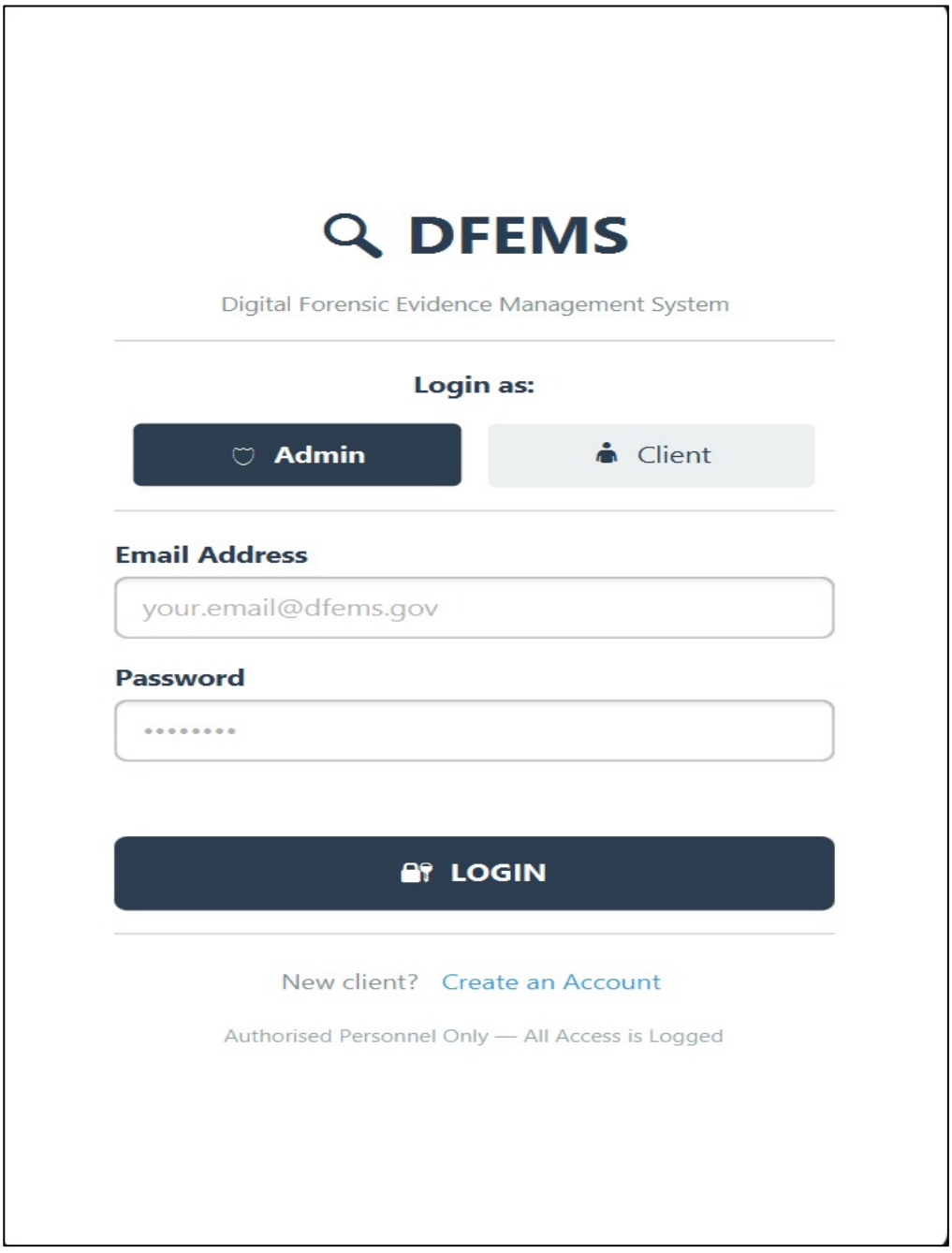


Fig : 1 : Login interface

Description: The login interface of the DFEMS system allows users to securely access the platform by selecting their role (Admin or Client) and entering credentials.

It ensures authorized access and provides an option for new users to create an account.

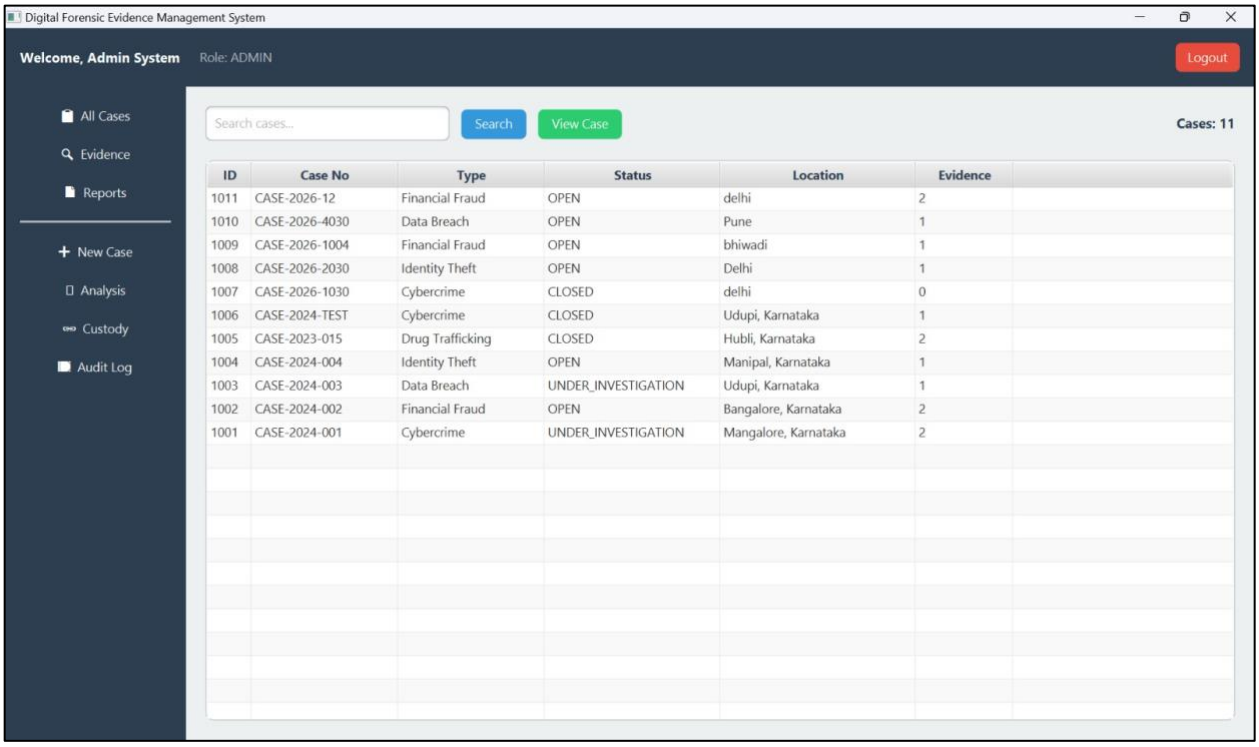


Fig : 2 : Admin Dashboard

Description: The dashboard provides an overview of all cases, displaying key details such as case ID, type, status, location, and associated evidence.

It allows users to search, view, and manage cases efficiently through a centralized interface.

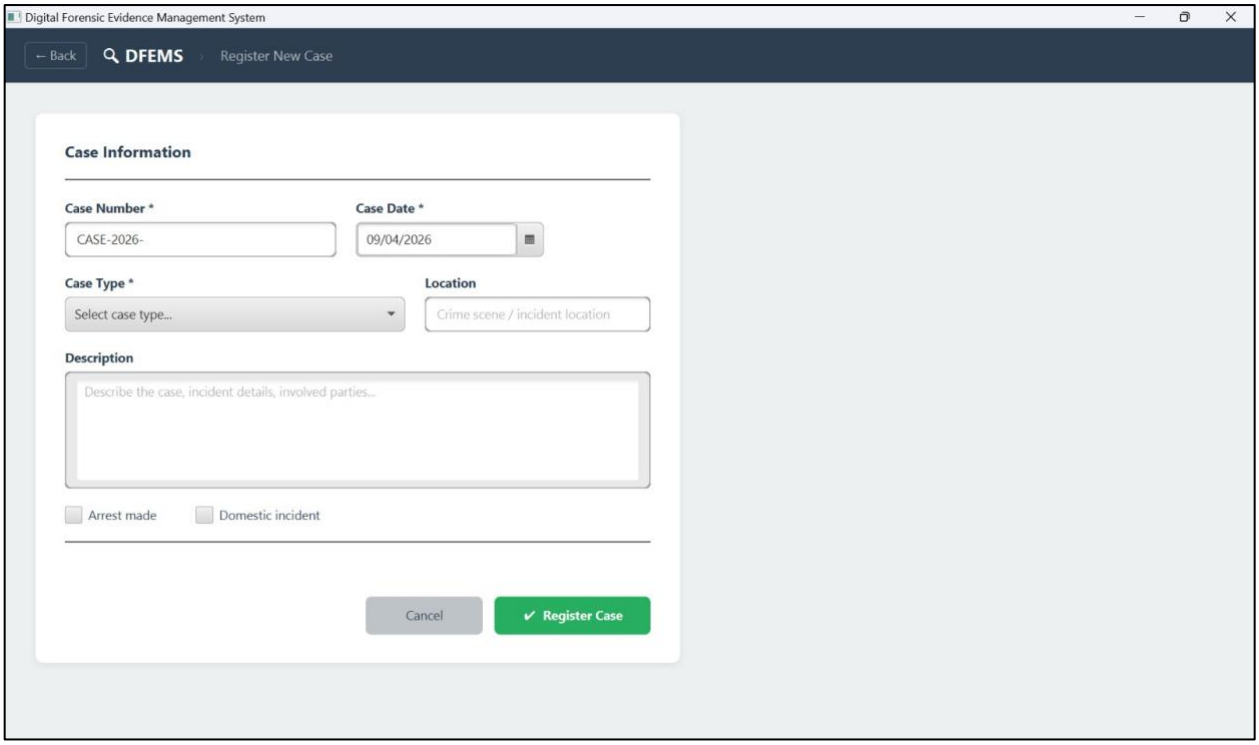


Fig : 3 : Register New Case Interface

Description: The case registration interface enables users to input detailed information such as case number, type, date, location, and description.

It ensures structured data entry for creating and managing new cases within the system.

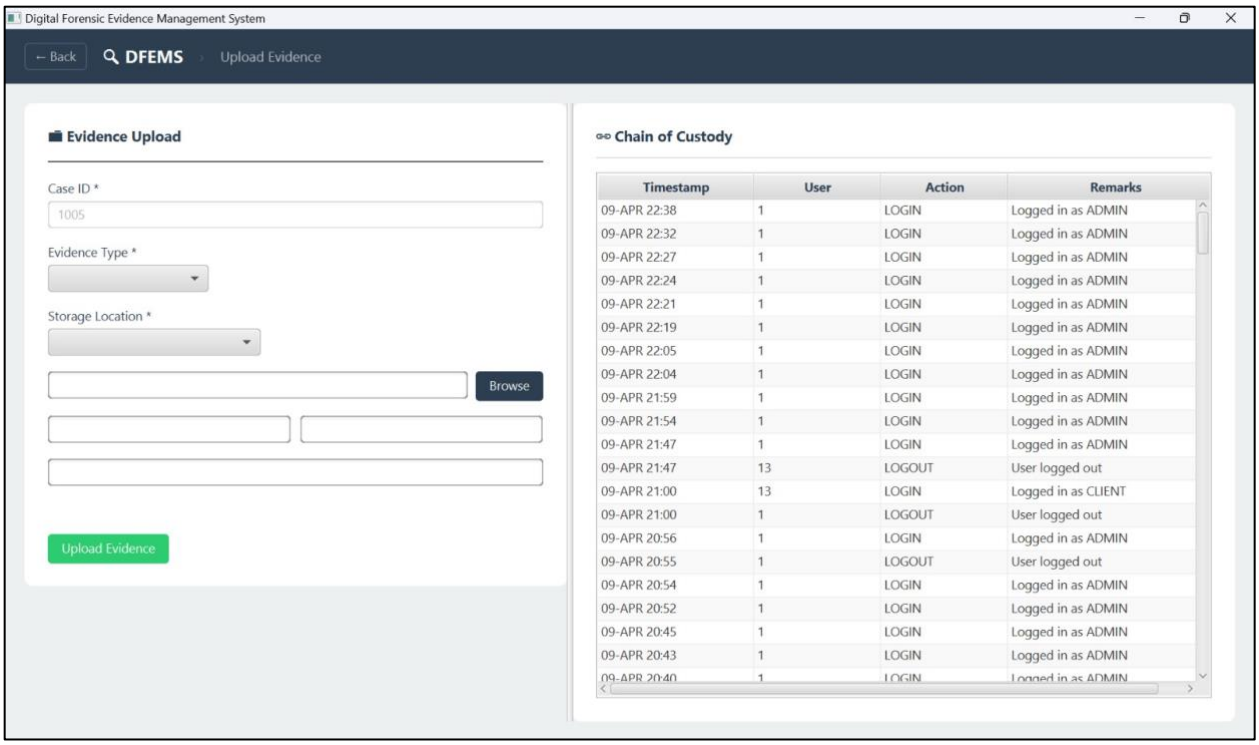


Fig : 4 : Evidence Upload Interface

Description: The evidence upload interface allows users to add digital evidence by specifying case details, file information, and storage location.

It also displays the chain of custody, providing a chronological record of user actions for maintaining evidence integrity.

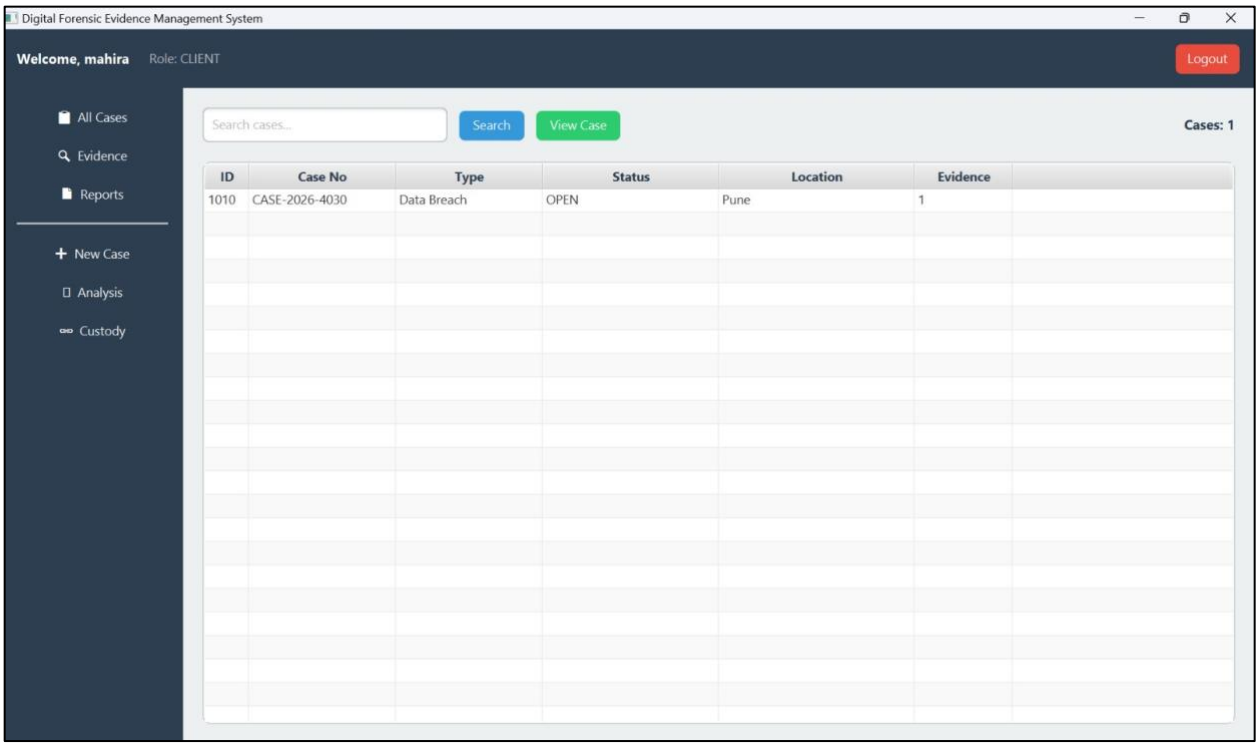


Fig : 5 : Client Dashboard

Description: The client dashboard provides a simplified view of assigned cases, displaying essential details such as case ID, type, status, and location.

It allows clients to search and view case information while restricting administrative functionalities.

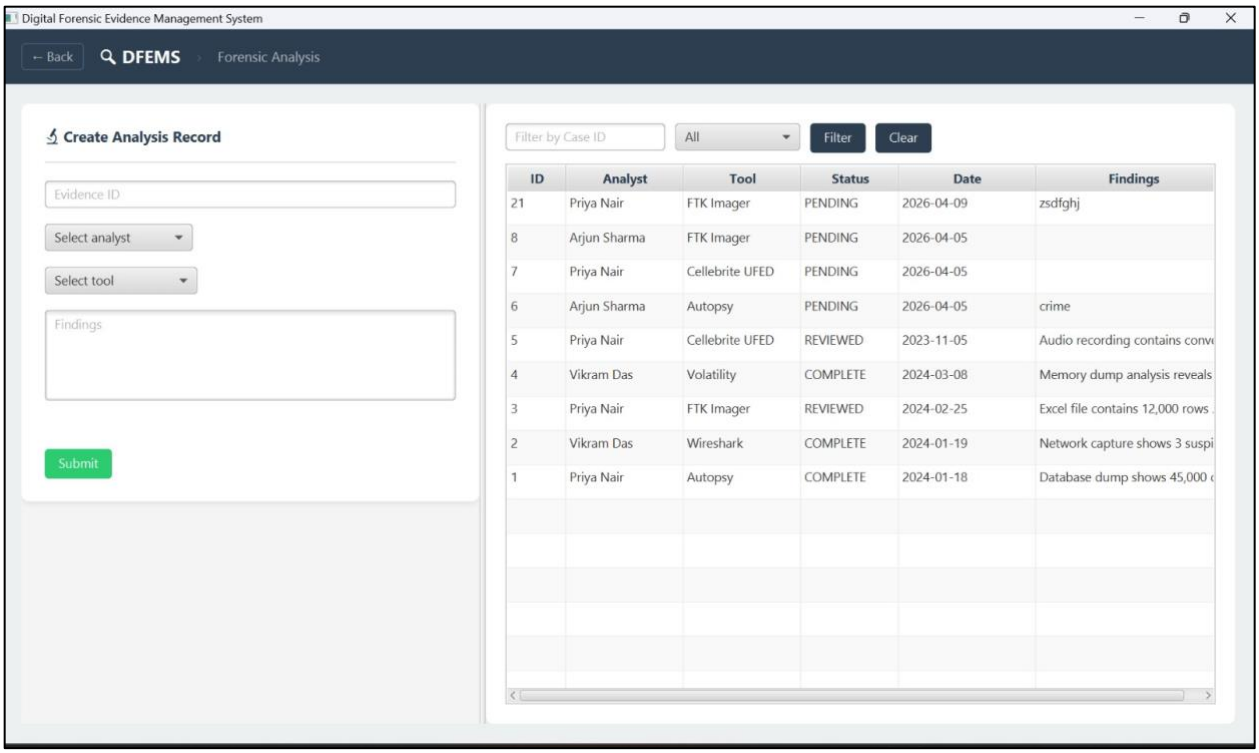


Fig : 6 : Analysis Page

Description: The forensic analysis interface allows users to create and manage analysis records by selecting evidence, analyst, tools, and entering findings.

It also displays a structured table of all analysis results with filtering options for efficient data retrieval

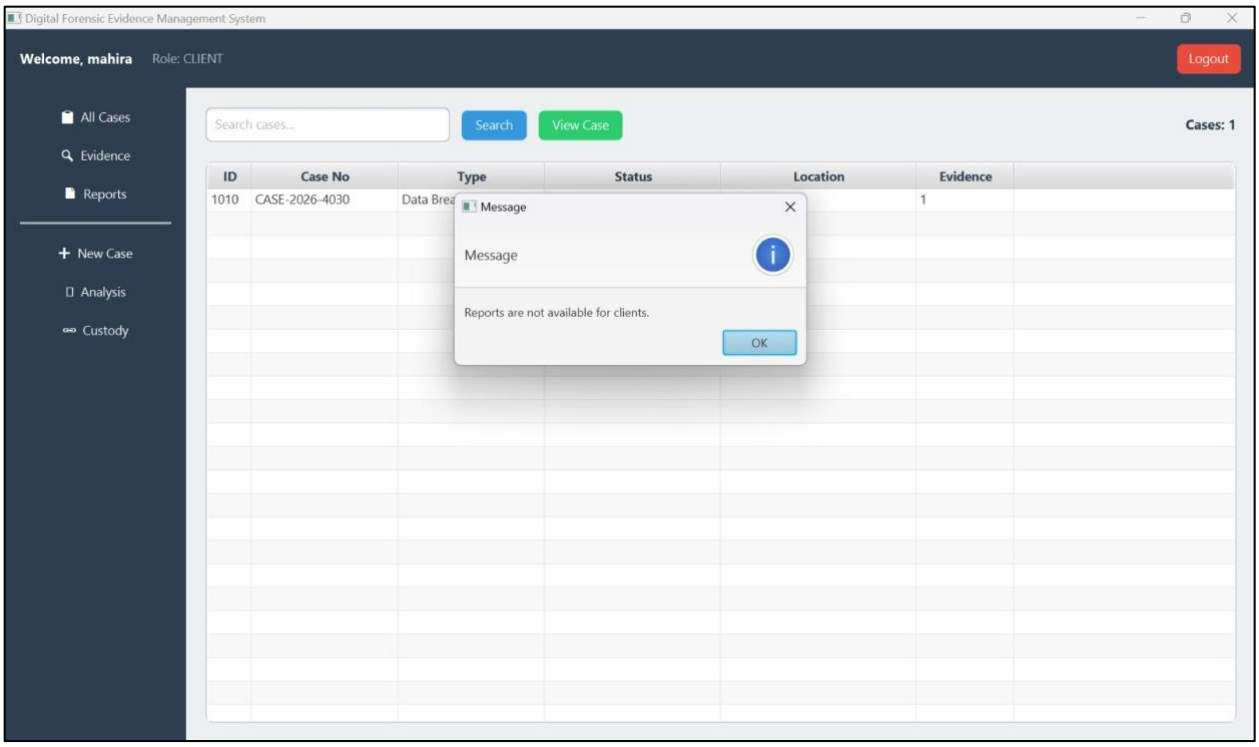


Fig : 7 : Client Dashboard

Description: The client dashboard restricts access to certain features, displaying a message when unauthorized actions like viewing reports are attempted.

It ensures role-based access control by limiting functionalities available to client users.

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

The Digital Forensic Evidence Management System (DFEMS) developed in this project demonstrates the practical implementation of database management system concepts in the domain of digital forensics. The system provides a structured and centralized platform for managing forensic cases, digital evidence, analysis records, and reports in an efficient and secure manner.

The database has been designed using proper normalization techniques, ensuring minimal redundancy and high data integrity. The use of primary and foreign key constraints establishes strong relationships among entities, enabling efficient data retrieval and consistency. Advanced features such as triggers, stored procedures, and audit logs have been incorporated to automate system processes and enhance reliability.

The system successfully implements role-based access control, allowing administrators and clients to interact with the system according to their permissions. Additionally, the chain of custody mechanism ensures traceability of evidence, which is a critical requirement in forensic investigations.

Overall, the project achieves its objective of designing a robust and scalable database system that effectively manages digital forensic data while demonstrating the real-world applicability of DBMS concepts.

7.2 Limitations

Despite its effectiveness, the current system has certain limitations:

- Lack of Complete PDF Report Generation:
  The system currently does not fully support automated generation of downloadable PDF reports.
- Limited Integration with External Tools:
  The system does not directly integrate with real-time forensic analysis tools.
- Basic User Interface:
  The current interface is functional but lacks advanced usability features and design enhancements.
- Restricted to Academic Scale:
  The system is designed for demonstration purposes and may require optimization for large-scale deployment.
- Limited Security Enhancements:
  Advanced security features such as encryption and multi-factor authentication are not fully implemented.

7.3 Future Work

The system can be further enhanced by incorporating the following improvements:

- **Automated PDF Report Generation:**
  Integration of reporting tools to generate structured and downloadable forensic reports.
- **Integration with Forensic Software:**
  Connecting the system with tools such as Autopsy or Wireshark for real-time analysis.
- **Improved User Interface:**
  Development of a more interactive and user-friendly graphical interface.
- **Cloud Deployment:**
  Hosting the system on cloud platforms to allow remote access and scalability.

- **Advanced Security Features**:
  Implementation of encryption, secure authentication mechanisms, and role-based access enhancements.
- **Real-Time Notifications:**
  Adding alerts for case updates, evidence uploads, and analysis completion.
- **Data Visualization and Analytics:**
  Incorporating dashboards and visualization tools for better interpretation of forensic data.

CHAPTER 8: REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] Oracle Corporation, “Oracle Database SQL Language Reference.” [Online]. Available: https://docs.oracle.com

[3] Oracle Corporation, “PL/SQL User’s Guide and Reference.” [Online]. Available: https://docs.oracle.com

[4] C. S. Horstmann and G. Cornell, Core Java Volume I – Fundamentals, 11th ed. Boston, MA, USA: Pearson, 2018.

[5] OpenJFX, “JavaFX API Documentation.” [Online]. Available: https://openjfx.io